

torch.kaiser_window#

https://pytorch.org/docs/stable/generated/torch.kaiser_window.html

Description:

Computes the Kaiser window with window length `window_length` and shape parameter `beta`. Let I_0 be the zeroth order modified Bessel function of the first kind (see `torch.i0()`) and $N = L - 1$ if `periodic` is `False` and L if `periodic` is `True`, where L is the `window_length`. This function computes: Calling `torch.kaiser_window(L, B, periodic=True)` is equivalent to calling `torch.kaiser_window(L + 1, B, periodic=False)[-1]`. The `periodic` argument is intended as a helpful shorthand to produce a periodic window as input to functions like `torch.stft()`. If `window_length` is one, then the returned window is a single element tensor containing a one.

Function Signature

```
torch.kaiser_window(window_length, periodic=True, beta=12.0,  
*, dtype=None, layout=torch.strided, device=None,  
requires_grad=False) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

`dtype` (`torch.dtype`, optional) – the desired data type of returned tensor. Default: if `None`, uses a global default (see `torch.set_default_dtype()`). `layout` (`torch.layout`, optional) – the desired layout of returned window tensor. Only `torch.strided` (dense layout) is supported. `device` (`torch.device`, optional) – the desired device of returned tensor. Default: if `None`, uses the current device for the default tensor type (see `torch.set_default_device()`). `device` will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types. `requires_grad` (`bool`, optional) – If autograd should record operations on the returned tensor. Default: `False`.

Notes & Warnings

NOTE

Note If `window_length` is one, then the returned window is a single element tensor containing a one.

Detailed Documentation

Documentation

Computes the Kaiser window with window length `window_length` and shape parameter

beta.

Let I_0 be the zeroth order modified Bessel function of the first kind (see `torch.i0()`) and $N = L - 1$ if `periodic` is `False` and L if `periodic` is `True, where  $L$  is the window_length. This function computes:`

Calling `torch.kaiser_window(L, B, periodic=True)` is equivalent to calling `torch.kaiser_window(L + 1, B, periodic=False)[-1]`. The `periodic` argument is intended as a helpful shorthand to produce a periodic window as input to functions like `torch.stft()`.

If `window_length` is one, then the returned window is a single element tensor containing a one.

`window_length` (int) – length of the window.

`periodic` (bool, optional) – If `True`, returns a periodic window suitable for use in spectral analysis. If `False`, returns a symmetric window suitable for use in filter design.

`beta` (float, optional) – shape parameter for the window.

`dtype` (`torch.dtype`, optional) – the desired data type of returned tensor. Default: if `None`, uses a global default (see `torch.set_default_dtype()`).

`layout` (`torch.layout`, optional) – the desired layout of returned window tensor. Only `torch.strided` (dense layout) is supported.

`device` (`torch.device`, optional) – the desired device of returned tensor. Default: if `None`, uses the current device for the default tensor type (see `torch.set_default_device()`). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types.

`requires_grad` (bool, optional) – If autograd should record operations on the returned tensor. Default: False.